

Spring Framework – Day 14 Notes

Bean Life Cycle with Collection Type Dependency Injection (List)

This session builds on the earlier bean life cycle and collection DI concepts by combining them in a single example. The **Hospital** bean demonstrates constructor, setter, init, and destroy callbacks together, while injecting a **List<String>** of treatments. The **HospitalService** bean is wired using **autowire="byType"** to automatically pick up a collection of Hospital beans, and demonstrates how **autowire-candidate="false"** can be used to control which beans participate in autowiring.

1. Hospital.java – Bean with Life Cycle Callbacks

File: *com/day14/Hospital.java*

```
package com.day14;

import java.util.List;

public class Hospital {

    private int hospitalCode;
    private String hospitalName;
    private String contactPerson;
    private long contactNumber;
    private List<String> treatment;

    private static int uniqueNumber = 1;

    public Hospital() {
        System.out.println("Hospital Class object created!");
    }

    public Hospital(String hospitalName) {
        this.hospitalName = hospitalName;
    }

    public void init() {
        System.out.println("Post constructio logic done");
        this.hospitalCode = uniqueNumber++;
    }

    public int getHospitalCode() {
        return hospitalCode;
    }

    public void setHospitalCode(int hospitalCode) {
        System.out.println("SETTER INJECTION DONE FOR HOSPITAL CLASS");
        this.hospitalCode = hospitalCode;
    }

    public String getHospitalName() {
        return hospitalName;
    }

    public void setHospitalName(String hospitalName) {
        System.out.println("SETTER INJECTION DONE FOR HOSPITAL CLASS");
        this.hospitalName = hospitalName;
    }

    public String getContactPerson() {
```

```

        return contactPerson;
    }

    public void setContactPerson(String contactPerson) {
        System.out.println("SETTER INJECTION DONE FOR HOSPITAL CLASS");
        this.contactPerson = contactPerson;
    }

    public long getContactNumber() {
        return contactNumber;
    }

    public void setContactNumber(long contactNumber) {
        System.out.println("SETTER INJECTION DONE FOR HOSPITAL CLASS");
        this.contactNumber = contactNumber;
    }

    public List<String> getTreatment() {
        return treatment;
    }

    public void setTreatment(List<String> treatment) {
        System.out.println("SETTER INJECTION DONE FOR HOSPITAL CLASS");
        this.treatment = treatment;
    }

    @Override
    public String toString() {
        return "Hospital [hospitalCode=" + hospitalCode + ", hospitalName=" + hospitalName + ", contactPerson="
            + contactPerson + ", contactNumber=" + contactNumber + ", treatment=" + treatment + "]";
    }

    public void destroy() {
        System.out.println("Post destruction logic executed");
        this.contactNumber=0;
        this.contactPerson="null";
        this.hospitalName="null";
        this.hospitalCode=0;
        this.treatment=null;
    }
}

```

Key points: **init()** runs after all properties are set (post-construction logic), assigning a unique hospital code using a static counter. **destroy()** runs just before the container shuts down, resetting all fields. Every setter also prints a confirmation message so the injection sequence is visible on the console.

2. HospitalService.java – Service Layer

File: *com/day14/HospitalService.java*

```
package com.day14;

import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class HospitalService {

    List<Hospital> data;

    public HospitalService() {

    }

    public List<Hospital> getData() {
        return data;
    }

    public void setData(List<Hospital> data) {
        this.data = data;
    }

    @Override
    public String toString() {
        return "HospitalService [data=" + data + "]";
    }

    public Map<String,Integer> getCodeAndName(){
        Map<String, Integer> result=new HashMap<String, Integer>();
        for(Hospital value:data) {
            result.put(value.getHospitalName(),
                value.getHospitalCode());
        }
        return result;
    }

    public Hospital getDetail(int hospitalCode) {
        Hospital result=null;
        for(Hospital value:data) {
            if(value.getHospitalCode()==hospitalCode) {
                result=value;
            }
        }
        return result;
    }

}
```

Key points: holds a **List<Hospital>** injected by the container. **getCodeAndName()** builds a Map of hospital name to hospital code. **getDetail(hospitalCode)** scans the list to find and return the matching Hospital object.

3. application-context.xml – Bean Configuration

File: *com/day14/application-context.xml*

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        https://www.springframework.org/schema/beans/spring-beans.xsd">
```

```

<bean id="hospital1" class="com.day14.Hospital" init-method="init"
    destroy-method="destroy">
    <property name="hospitalName" value="Appolo"></property>
    <property name="contactPerson" value="abc"></property>
    <property name="contactNumber" value="12341414"></property>
    <property name="treatment">
        <list>
            <value>ENT</value>
            <value>Cardic</value>
        </list>
    </property>
</bean>

<bean id="hospital2" class="com.day14.Hospital" init-method="init"
    destroy-method="destroy" autowire-candidate="false">
    <property name="hospitalName" value="Yashoda"></property>
    <property name="contactPerson" value="def"></property>
    <property name="contactNumber" value="54523523"></property>
    <property name="treatment">
        <list>
            <value>Gastro</value>
            <value>Neuro</value>
        </list>
    </property>
</bean>

<bean id="hospital3" class="com.day14.Hospital" init-method="init"
    destroy-method="destroy" autowire-candidate="false">
    <property name="hospitalName" value="Aster-Prime"></property>
    <property name="contactPerson" value="xyz"></property>
    <property name="contactNumber" value="123123"></property>
    <property name="treatment">
        <list>
            <value>Genral</value>
            <value>Dental</value>
        </list>
    </property>
</bean>

<bean id="hospitalService" class="com.day14.HospitalService"
    autowire="byType">
</bean>
</beans>

```

Key points: three Hospital beans are declared, each with **init-method="init"** and **destroy-method="destroy"**. hospital2 and hospital3 use **autowire-candidate="false"**, meaning they are excluded from autowiring by type — only hospital1 remains eligible. The **hospitalService** bean uses **autowire="byType"**, so Spring looks for a matching type (List of Hospital / Hospital beans) among autowire-eligible beans to inject into its **data** property.

4. Main.java – Application Entry Point

File: com/day14/Main.java

```
package com.day14;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Main {
    public static void main(String[] args) {
        ApplicationContext context=new
            ClassPathXmlApplicationContext
                ("com/day14/application-context.xml");

        HospitalService bean = context.getBean("hospitalService",HospitalService.class);

        System.out.println(bean.getCodeAndName());

        Hospital bean2 = context.getBean("hospital1", Hospital.class);
        int code=bean2.getHospitalCode();
        Hospital detail=bean.getDetail(code);
        if(detail!=null) {
            System.out.println(detail);
        }
        else {
            System.out.println("No matching found!");
        }
    }
}
```

Key points: loads the Spring container from **application-context.xml**, retrieves the **hospitalService** bean, and prints the code-to-name map. It then fetches **hospital1** directly, reads its generated hospital code, and looks up the matching detail through the service layer.

Summary

- **Life cycle order:** Constructor → Setter Injection → init-method → (bean ready for use) → destroy-method (on container shutdown).
- **Collection DI:** a <list> element inside <property> injects a List<String> of treatment values.
- **autowire="byType":** Spring matches a bean property's type against available autowire-candidate beans in the container.
- **autowire-candidate="false":** excludes a bean from being chosen automatically during byType/byName autowiring, without removing it from the container.